REPORT - MATHEMATICAL INTELLIGENCE TOOLKIT

AUTHOR - Ms. Siddhima Singh

COURSE - Fundamentals to AI/ML

Date - 03/04/26

1. Introduction

This project presents a Mathematical Intelligence Toolkit that applies algorithmic logic and computational techniques to solve numerical problems efficiently.

The project demonstrates how structured mathematical reasoning, pattern recognition, and algorithm design—key components in Artificial Intelligence systems—can be implemented using Python.

2. Objective

The objective of this project is to develop a system that performs various number-theoretic computations while showcasing the foundational logic behind intelligent systems used in AI and Machine Learning.

3. IMPLEMENTATION DETAILS

This report outlines 34 comprehensive functions covering fundamental to advanced topics in number theory.

integer_sqrt(n):

Purpose: Finds the whole number part of the square root of a given number 'n'.

Methodology: Calculates the root and then removes any fractional part, leaving only the integer.

gcd(a, b):

Purpose: Determines the largest positive integer that divides both input integers, 'a' and 'b', without leaving a remainder. This is known as the Greatest Common Divisor.

Methodology: Uses an efficient, repetitive division process called the Euclidean algorithm.

get_integer_log2(n):

Purpose: Finds out how many times the number 2 can be multiplied by itself to get a value that is less than or equal to 'n'.

Methodology: Continuously multiplies the number 2, counting the steps until the resulting value exceeds 'n'.

power(base, exp):

Purpose: Calculates the result of multiplying a 'base' number by itself a specified number of times ('exp').

Methodology: Repeats multiplication of the base number for the number of times indicated by the exponent.

factorial(n):

Purpose: Computes the product obtained by multiplying all whole numbers from 1 up to the number 'n'.

Methodology: Iteratively multiplies the sequence of integers starting from 1 up to 'n'.

is_palindrome(n):

Purpose: Checks if a number 'n' remains the same when its digits are read in reverse order.

Methodology: Converts the number into a sequence of characters and checks if the sequence is identical to its reverse.

mean_of_digits(n):

Purpose: Calculates the average value of all the individual digits that make up the number 'n'.

Methodology: Adds up all the digits and then divides the total by the count of how many digits there are.

digital_root(n):

Purpose: Finds the single-digit value that results from repeatedly adding the digits of a number until only one digit remains.

Methodology: Uses a mathematical shortcut based on remainder when dividing by nine, which is a very fast way to find the result.

aliquot_sum(n):

Purpose: Calculates the total value when all the proper divisors of 'n' are added together. Proper divisors are all divisors of 'n' except 'n' itself.

Methodology: Systematically finds all numbers that divide 'n' evenly (excluding 'n') and sums them up.

is_abundant(n):

Purpose: Checks if the sum of the proper divisors of 'n' is larger than the number 'n' itself.

Methodology: Compares the result from the aliquot sum function with the original number 'n'.

is_deficient(n):

Purpose: Checks if the sum of the proper divisors of 'n' is smaller than the number 'n' itself.

Methodology: Compares the result from the aliquot sum function with the original number 'n'.

is_harshad(n):

Purpose: Checks if 'n' can be perfectly divided by the sum of its own digits.

Methodology: Calculates the sum of the digits and then performs a divisibility test on 'n'.

is_automorphic(n):

Purpose: Checks if a number 'n' appears as the final digits of its own square.

Methodology: Squares the number and then checks if the squared result ends with the exact sequence of digits as the original number 'n'.

is_pronic(n):

Purpose: Checks if 'n' can be formed by multiplying two consecutive whole numbers together.

Methodology: Calculates the whole number square root of 'n' and checks if that number multiplied by the next consecutive number equals 'n'.

is_prime_trial_division(n):

Purpose: Determines if 'n' is a prime number (only divisible by 1 and itself) using a basic testing method.

Methodology: Systematically checks for division by smaller numbers up to the square root of 'n'.

prime_factors(n):

Purpose: Finds all the prime numbers that, when multiplied together, equal 'n'. This includes repeated factors.

Methodology: Repeatedly divides 'n' by the smallest possible prime factors until only 1 remains.

count_distinct_prime_factors(n):

Purpose: Counts the total number of unique prime numbers that divide 'n'.

Methodology: Identifies all prime factors of 'n' and then counts how many different primes appear in that list.

is_prime_power(n):

Purpose: Checks if 'n' is the result of a single prime number multiplied by itself one or more times.

Methodology: Examines the prime factors of 'n' to ensure they are all the same number.

is_mersenne_prime(p):

Purpose: Checks if a number one less than a power of two is a prime number, where the exponent 'p' is itself a prime.

Methodology: Calculates the value and then runs a primality test on the result.

twin_primes(limit):

Purpose: Identifies all pairs of prime numbers that are separated by a difference of exactly two, up to a specified maximum value.

Methodology: Uses an efficient method to find all prime numbers and then checks which ones form pairs with a gap of two.

count_divisors(n):

Purpose: Counts the total number of positive integers that divide 'n' evenly.

Methodology: Iterates through possible divisors up to the square root of 'n' to find and count all pairs of divisors.

get_aliquot_sum:

Purpose: A different name for the function that calculates the sum of the proper divisors of 'n'.

Methodology: Same as function 9.

are_amicable(a, b):

Purpose: Checks if two different numbers, 'a' and 'b', are an amicable pair. This means the sum of the proper divisors of 'a' equals 'b', and the sum of the proper divisors of 'b' equals 'a'.

Methodology: Calls the aliquot sum function for both numbers and compares the results to ensure they match the other number.

multiplicative_persistence(n):

Purpose: Determines how many times you must multiply the digits of a number to reduce it down to a single-digit number.

Methodology: Repeatedly calculates the product of the number's digits until the resulting number is less than ten, counting the steps taken.

is_highly_composite(n):

Purpose: Checks if 'n' has more divisors than any other whole number smaller than 'n'.

Methodology: Compares the total count of divisors for 'n' against the highest divisor count found among all preceding numbers.

mod_exp(base, exponent, modulus):

Purpose: Calculates the remainder when a 'base' number, raised to an 'exponent', is divided by a 'modulus'.

Methodology: Uses a highly efficient technique called exponentiation by squaring to keep the intermediate numbers small throughout the calculation.

extended_gcd(a, b):

Purpose: Finds the Greatest Common Divisor of 'a' and 'b' and also finds two related integers, 'x' and 'y', that satisfy a specific algebraic relationship with 'a' and 'b'.

Methodology: Uses a variation of the Euclidean algorithm that keeps track of the necessary coefficients.

mod_inverse(a, m):

Purpose: Finds the modular multiplicative inverse of 'a' relative to 'm'. This is a number that, when multiplied by 'a', leaves a remainder of 1 after being divided by 'm'.

Methodology: Relies on the result of the extended GCD function to solve for the inverse.

crt(remainders, moduli):

Purpose: Solves a system of equations where a single unknown number leaves specified 'remainders' when divided by a list of 'moduli' (divisors). This is the Chinese Remainder Theorem.

Methodology: Progressively combines the conditions from the system of equations to find the final solution.

is_quadratic_residue(a, p):

Purpose: Checks if the number 'a' is a quadratic residue modulo 'p'. This means checking if 'a' is a perfect square when working within the set of remainders created by dividing by 'p'.

Methodology: Uses Euler's Criterion, which tests the result of a modular exponentiation to determine if a solution exists.

order_mod(a, n):

Purpose: Finds the multiplicative order of 'a' modulo 'n'. This is the smallest number of times 'a' must be multiplied by itself to produce a remainder of 1 when divided by 'n'.

Methodology: Iteratively calculates the powers of 'a' and checks the remainder until 1 is reached.

is_fibonacci_prime(n):

Purpose: Checks if a prime number 'n' is also a term in the famous Fibonacci sequence.

Methodology: Uses a special mathematical property where a number is a Fibonacci term if a related calculation involving its square results in a perfect square.

lucas_sequence(n):

Purpose: Generates the first 'n' terms of the Lucas sequence, which is similar to Fibonacci but starts with 2 and 1.

Methodology: Creates the sequence by summing the two preceding terms, beginning with 2 and 1.

is_perfect_power(n):

Purpose: Checks if 'n' can be written as another whole number raised to an integer power greater than one.

Methodology: Systematically tests possible exponents (like 2, 3, 4, etc.) to see if taking that root of 'n' results in a whole number.

4. Learning Outcomes

- Developed strong algorithmic thinking
- Understood computational logic and optimization
- Explored mathematical foundations relevant to AI systems

5. Conclusion

This project highlights how mathematical logic and algorithmic structures contribute to the foundation of Artificial Intelligence and Machine Learning systems. It demonstrates the importance of efficient computation and structured problem-solving in building intelligent applications.